

Université de MOULOUD MAMMERI de TIZI-OUZOU

Faculté de génie électrique et de l'informatique

Département d'Informatique



Projet : "système de questions-réponses (QA - Question Answering)"

Module : Traitement Automatique du Langage Naturel

Réalisé par :

- **GUEDOUAR Ouassila**
- **GUEDOUAR Sonia**
- **MORSLI Manel**

Promotion : 2024/2025

SOMMAIRE

I. Introduction	4
II. Présentation de la tâche et des objectifs:	4
III. Présentation des données:	5
IV. Méthodologie:	5
1. chargement des données :	5
2. Prétraitement:	6
A. Installation des bibliothèques essentielles.	6
B. Importation des modules nécessaires.	7
C. Charger en DataFrame.	7
D. Nettoyer les données.	7
3. Division dataset:	8
4. Fine Tuning.	9
5. Évaluation et test du modèle.	9
V. Perspectives.	9
VI. conclusion.	9

I. Introduction

Dans un contexte où les étudiants cherchent à obtenir rapidement des informations sur leurs emplois du temps, leurs salles de cours, leurs modules, leurs enseignants,...etc. Les systèmes de **questions-réponses (QA - Question Answering)** basés sur l'intelligence artificielle deviennent une solution efficace et intuitive.

Ces systèmes permettent de traiter des requêtes en langage naturel et de fournir des réponses précises en s'appuyant sur des modèles de traitement du langage naturel (NLP).

Ce projet vise à **fine-tuner le modèle GPT-2** afin de l'adapter spécifiquement aux besoins des étudiants en lui fournissant des données propres à leur département.

Dans ce rapport, nous présenterons d'abord les objectifs du projet et les données utilisées. Ensuite, nous détaillerons la méthodologie suivie, en abordant les étapes de collecte et de prétraitement des données, le choix du modèle et son entraînement. Enfin, nous évaluerons les performances du système avant de conclure sur les résultats obtenus et les perspectives d'amélioration.

II. Présentation de la tâche et des objectifs:

Ce projet vise à concevoir un **système de questions-réponses (QA)** permettant aux étudiants de notre département d'accéder rapidement à des informations essentielles concernant leur **emploi du temps, leurs modules, les enseignants et la disponibilité des salles**.

Pour atteindre cet objectif, nous utilisons le modèle de langue **GPT-2**, que nous fine-tunons sur un ensemble de **données spécifiques à notre département d'informatique**.

Afin de valider les performances de notre approche, une évaluation est faite, voici les principales étapes du projet :

- **Collecte et structuration des données** liées au département d'informatique.
- **Fine-tuning de GPT-2** sur ces données.
- **Évaluation des performances** du modèle.
- **Tests et analyse des résultats** pour valider l'efficacité du système.

III. Présentation des données:

Les performances d'un modèle de NLP reposent en grande partie sur la qualité et la pertinence des données utilisées pour son entraînement.

Nous avons extrait les données des plannings académiques de différents niveaux et spécialités du département d'informatique. Pour les structurer dans un format exploitable par le modèle

de langue choisi, nous avons transformé les informations issues des **images des plannings** en un jeu de données organisé .

Cette structuration permet au modèle d'accéder rapidement aux informations demandées, garantissant ainsi un service parfaitement adapté aux besoins des étudiants.

pour représenter nos données sous le format manipulable on a commencé par:

1. Collecte de données:

La première étape du projet consistait à **recupérer et organiser les données** afin de les structurer. Cette phase inclut :

- **L'extraction des informations** à partir des différents plannings académiques(images).
- **La conversion des données** dans un format structuré (CSV) et compatible pour l'entraînement du modèle .
- **La vérification et la correction** des éventuelles incohérences pour assurer la qualité des données.

Cette structuration nous a permis d'obtenir un jeu de données adapté pour les étapes qui suivent.

IV. Méthodologie:

Dans cette partie nous allons détailler le processus de création du système QA

1. chargement des données :

On charge notre fichier intitulé "**augmented dep.csv**" sur google colab afin de le convertir en dataframe,pour pouvoir effectuer les prétraitements nécessaires sur nos données .

```
[ ] from google.colab import files
    uploaded=files.upload()
```



Sélect. fichiers augmented_dep.csv

- **augmented_dep.csv**(text/csv) - 662109 bytes, last modified: 02/02/2025 - 100% done
Saving augmented_dep.csv to augmented_dep.csv

2. Prétraitement:

Cette étape est cruciale pour garantir la performance de n'importe quel modèle.

Une fois le fichier est chargé, les données doivent être nettoyées et préparées pour une utilisation ultérieure.

A. Installation des bibliothèques essentielles:

Transformers : pour les modèles de traitement du langage naturel comme GPT-2, BERT, et LLaMA.

PyTorch : pour l'apprentissage profond et les réseaux de neurones.

NLTK (Natural Language Toolkit) : pour le traitement du langage naturel, y compris le tokenization, la stemming et le tagging.

NumPy : pour les calculs numériques et les opérations sur les tableaux.

Pandas : pour la manipulation des données; les DataFrames.

re : pour la manipulation des expressions régulières et les règles de correspondance de motifs.

datasets: charger un dataset depuis Hugging Face.

```
!pip install transformers
!pip install torch
!pip install tqdm
!pip install nltk

Requirement already satisfied: transformers in /usr/local/lib/python3.11/dist-packages (4.47.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-packages (from transformers) (3.17.0)
Requirement already satisfied: huggingface-hub<1.0,>=0.24.0 in /usr/local/lib/python3.11/dist-packages (from transformers) (0.24.0)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.11/dist-packages (from transformers) (1.26.4)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/dist-packages (from transformers) (24.1)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.11/dist-packages (from transformers) (6.0.2)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.11/dist-packages (from transformers) (2024.9.11)
Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packages (from transformers) (2.32.3)
Requirement already satisfied: tokenizers<0.22,>=0.21 in /usr/local/lib/python3.11/dist-packages (from transformers) (0.21.0)
```

B. Importation des modules nécessaires:

```
import pandas as pd
import numpy as np
import re
import nltk
from sklearn.model_selection import train_test_split
from transformers import GPT2LMHeadModel, GPT2Tokenizer, AdamW
import torch
from torch.utils.data import Dataset, DataLoader
from tqdm import tqdm
import json

nltk.download('punkt')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt.zip.
True
```

C. Charger en DataFrame :

le fichier csv est d'abord converti en un dataframe pour faciliter sa manipulation

notre dataframe se compose de 3 colonnes : "question", "answer" et "tags".

On charge notre fichier CSV, puis on applique un prétraitement sur les colonnes **question** et **answer** en supprimant les caractères # et ", et en convertissant le texte en minuscules.

<pre># Fonction de prétraitement def preprocess_text(text): text = re.sub(r'["]', '', text) # On Supprime uniquement les caractères # et " return text.lower() # Charger et appliquer le pretraitement sur les colonnes df = pd.read_csv('augmented_dep.csv') df['question'] = df['question'].apply(preprocess_text) df['answer'] = df['answer'].apply(preprocess_text) df</pre>			
	question	answer	tags
0	où se fait le cours de informatique ?	le cours se fait à amphi b.	lieu, informatique
1	qui enseigne informatique ?	y. bouhadjar enseigne ce cours.	professeur
2	a quelle heure commence le cours de informatiq...	le cours commence à 08h00-10h00.	horaires, informatique
3	où se fait le cours de mathématiques ?	le cours se fait à amphi b.	lieu, mathématiques
4	qui enseigne mathématiques ?	k. massinissa enseigne ce cours.	professeur
...	...	...	...
8200	quel jour a lieu le cours de anglais iii ?	jeudi	anglais

3. Division dataset:

Pour entraîner notre modèle de manière efficace et évaluer ses performances, nous avons divisé notre dataset de manière aléatoire mais reproductible grâce à random state=42 en 2:

- ensemble d'entraînement 90% du dataset.
- ensemble de validation 10% du dataset.

4. Fine Tuning:

Le fine-tuning est une technique permettant de spécialiser un modèle pré-entraîné sur une tâche bien spécifique.

- ❖ **Le choix du modèle pré-entraîné:** Le choix du modèle dépend de la tâche; pour notre cas: générer des réponses précises à des questions posées, le mieux adapté est GPT car :

1. Il utilise une architecture de type transformers génératif pré-entraîné (GPT). C'est un modèle basé sur un **décodeur** (Transformer) qui prédit le texte **mot par mot** en s'appuyant sur les tokens précédents.

2. Il génère du texte fluide et cohérent, utilisé pour des tâches comme la **conversation, la complétion de texte, et la génération de contenu**.

3. **Adapté pour** : Chatbots, génération de réponses...etc.

GPT est un modèle de langue particulièrement puissant grâce à :

- Sa flexibilité pour s'adapter à différentes tâches NLP via le fine-tuning.
- Son efficacité pour comprendre et répondre aux questions en langage naturel.

- **Fine-tuning sur nos données:**

Le fine-tuning a été réalisé en utilisant les données collectées, avec un ajustement des hyperparamètres pour optimiser les performances du modèle. Les étapes incluent :

- Chargement du modèle pré-entraîné GPT-2.
- Adaptation du format des données pour l'entraînement.
- Entraînement du modèle avec des itérations successives.
- Évaluation des performances sur un ensemble de validation.

pour se faire on suit les étapes suivantes:

- On a créé un QADataset pour préparer les données de question-réponse .

Elle prend en entrée les questions, les réponses, un tokenizer et une longueur maximale des séquences. La méthode `__len__` retourne la taille du dataset, tandis que `__getitem__` encode chaque paire question-réponse en utilisant le tokenizer, avec padding et troncature si nécessaire. Elle retourne ensuite un dictionnaire contenant les `input_ids`, le `attention_mask` et les labels, qui seront utilisés pour entraîner un modèle de génération de texte ou de classification.

```
# Créer une classe Dataset personnalisée
class QADataset(Dataset):
    def __init__(self, questions, answers, tokenizer, max_len):
        self.questions = questions
        self.answers = answers
        self.tokenizer = tokenizer
        self.max_len = max_len

    def __len__(self):
        return len(self.questions)

    def __getitem__(self, index):
        question = str(self.questions[index])
        answer = str(self.answers[index])
        encoding = self.tokenizer.encode_plus(
            question,
            answer,
            max_length=self.max_len,
            padding='max_length',
            truncation=True,
            return_tensors='pt'
        )
        return {
            'input_ids': encoding['input_ids'].flatten(),
            'attention_mask': encoding['attention_mask'].flatten(),
            'labels': encoding['input_ids'].flatten()
        }
```

✓ Connecté à Backend Google Colab

- On a créé un **DataLoader** pour charger les données d'entraînement en lots (batches).

✓
1 s

```
# Créer le DataLoader
def create_data_loader(df, tokenizer, max_len, batch_size):
    ds = QADataset(
        questions=df.question.to_numpy(),
        answers=df.answer.to_numpy(),
        tokenizer=tokenizer,
        max_len=max_len
    )
    return DataLoader(ds, batch_size=batch_size, num_workers=4)
```

- On a chargé le tokenizer GPT-2, ensuite, on a défini le token de padding comme le token de fin de séquence, puis on a créé des DataLoaders pour les ensembles d'entraînement et de validation en utilisant le tokenizer et une taille de séquence maximale de 128. On configure l'entraînement en sélectionnant le périphérique (GPU ou CPU) pour l'entraînement et enfin on charge le modèle GPT-2 pré-entraîné pour l'entraînement.


```
✓ 12 s
tokenizer = GPT2Tokenizer.from_pretrained('gpt2')
tokenizer.pad_token = tokenizer.eos_token # Utiliser le token de fin comme pad_token

train_data_loader = create_data_loader(train_data, tokenizer, max_len=128, batch_size=8)
val_data_loader = create_data_loader(val_data, tokenizer, max_len=128, batch_size=8)

# Configurer l'entraînement
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = GPT2LMHeadModel.from_pretrained('gpt2')
model = model.to(device)

/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens),
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
tokenizer_config.json: 100% ██████████ 26.0/26.0 [00:00<00:00, 648B/s]
vocab.json: 100% ██████████ 1.04M/1.04M [00:00<00:00, 18.5MB/s]
merges.txt: 100% ██████████ 456k/456k [00:00<00:00, 10.3MB/s]
tokenizer.json: 100% ██████████ 1.36M/1.36M [00:00<00:00, 1.57MB/s]
config.json: 100% ██████████ 665/665 [00:00<00:00, 45.4kB/s]
```

- On a configuré l'optimisation et on a ajusté le taux d'apprentissage pour l'entraînement du modèle.
Un optimiseur **AdamW** est créé avec un taux d'apprentissage de **2e-5**, ce qui permet d'ajuster les paramètres du modèle, et l'option **correct_bias=False** désactive la correction du biais, couramment utilisée avec les modèles pré-entraînés.
Ensuite, un planificateur d'apprentissage est configuré à l'aide de **get_linear_schedule_with_warmup**, qui ajuste linéairement le taux d'apprentissage pendant l'entraînement. Il commence par un faible taux d'apprentissage, l'augmente progressivement (bien que sans phase de réchauffement ici, car `num_warmup_steps=0`), puis le réduit linéairement tout au long des étapes d'entraînement, pour un total de 3 époques.

```
✓ 1 s
from transformers import get_linear_schedule_with_warmup

optimizer = AdamW(model.parameters(), lr=2e-5, correct_bias=False)
total_steps = len(train_data_loader) * 3 # Nombre d'époques
scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=0,
    num_training_steps=total_steps
)

/usr/local/lib/python3.11/dist-packages/transformers/optimization.py:591: FutureWarning: This
warnings.warn(
```

- La fonction **train_epoch** gère l'entraînement d'une époque pour le modèle. Elle commence par placer le modèle en mode entraînement avec `model.train()`, ce qui permet de mettre à jour les paramètres du modèle pendant l'entraînement. Ensuite, elle initialise une liste pour stocker les pertes à chaque étape. Le modèle est utilisé pour calculer la perte, puis la rétropropagation est effectuée avec **loss.backward()**, suivie de la mise à jour des poids avec **optimizer.step()**. Le taux d'apprentissage est ajusté avec **scheduler.step()**, et les gradients sont réinitialisés avec **optimizer.zero_grad()**.

Enfin, la fonction retourne la moyenne des pertes calculées pendant l'entraînement, offrant ainsi un indicateur de la performance du modèle pour cette époque.

```
✓ 0s ▶ def train_epoch(model, data_loader, optimizer, device, scheduler):  
    model = model.train()  
    losses = []  
    for d in tqdm(data_loader):  
        input_ids = d['input_ids'].to(device)  
        attention_mask = d['attention_mask'].to(device)  
        labels = d['labels'].to(device)  
  
        outputs = model(  
            input_ids=input_ids,  
            attention_mask=attention_mask,  
            labels=labels  
        )  
        loss = outputs.loss  
        losses.append(loss.item())  
        loss.backward()  
  
        optimizer.step()  
        scheduler.step()  
        optimizer.zero_grad()  
    return np.mean(losses)
```

- **eval_model :**

La fonction **eval_model** évalue la performance du modèle sans modifier ses paramètres. Le modèle est placé en mode évaluation avec **model.eval()**, ce qui désactive des mécanismes comme le **dropout**. La fonction itère ensuite sur les lots de données du **data_loader**, et les **input_ids**, **attention_mask**, et **labels** sont transférés sur le périphérique.

La perte est calculée en appelant le modèle, et elle est ajoutée à la liste des pertes.

Le **with torch.no_grad()** désactive le calcul des gradients, ce qui économise de la mémoire et accélère l'évaluation.

```

def eval_model(model, data_loader, device):
    model = model.eval()
    losses = []
    with torch.no_grad():
        for d in data_loader:
            input_ids = d['input_ids'].to(device)
            attention_mask = d['attention_mask'].to(device)
            labels = d['labels'].to(device)

            outputs = model(
                input_ids=input_ids,
                attention_mask=attention_mask,
                labels=labels
            )
            loss = outputs.loss
            losses.append(loss.item())
    return np.mean(losses)

```

- Ici, on entraîne le modèle GPT-2 pendant 3 époques. À chaque époque, il affiche le numéro de l'époque et calcule la perte d'entraînement en appelant la fonction **train_epoch**. Ensuite, on évalue le modèle sur les données de validation en calculant la perte de validation avec la fonction **eval_model**.

Les pertes d'entraînement et de validation sont affichées pour chaque époque. Une fois l'entraînement terminé, le modèle fine-tuné et le tokenizer sont sauvegardés dans un dossier nommé **fine_tuned_gpt2**.

```
✓ 13 min ▶ EPOCHS = 3

for epoch in range(EPOCHS):
    print(f'Epoch {epoch + 1}/{EPOCHS}')
    train_loss = train_epoch(model, train_data_loader, optimizer, device, scheduler)
    print(f'Train loss {train_loss}')
    val_loss = eval_model(model, val_data_loader, device)
    print(f'Val loss {val_loss}')

# Sauvegarder le modèle fine-tuné et le tokenizer
model.save_pretrained('fine_tuned_gpt2')
tokenizer.save_pretrained('fine_tuned_gpt2')
```

↩ Epoch 1/3
100% ██████████ 934/934 [04:27<00:00, 3.49it/s] Train loss 0.15461977422077855
Val loss 0.07031436619133903
Epoch 2/3
100% ██████████ 934/934 [04:28<00:00, 3.48it/s] Train loss 0.08335167100569293
Val loss 0.06094487960665272
Epoch 3/3
100% ██████████ 934/934 [04:28<00:00, 3.47it/s] Train loss 0.07599548347757
Val loss 0.05888700581943759
(
 'fine_tuned_gpt2/tokenizer_config.json',
 'fine_tuned_gpt2/special_tokens_map.json',
 'fine_tuned_gpt2/vocab.json',
 'fine_tuned_gpt2/merges.txt',
 'fine_tuned_gpt2/added_tokens.json')

- On a créé deux DataFrames à partir des données d'entraînement (**train_data**) et de validation (**val_data**), en extrayant uniquement les colonnes "question" et "answer". Ensuite, on sauvegarde ces DataFrames sous forme de fichiers JSON. Ces fichiers JSON sont ensuite prêts à être utilisés pour l'entraînement ou l'évaluation du modèle.

```
✓ 1 s ▶

# Sauvegarder les embeddings en JSON
train_df = pd.DataFrame({
    'question': train_data['question'],
    'answer': train_data['answer'],
})

val_df = pd.DataFrame({
    'question': val_data['question'],
    'answer': val_data['answer'],
})

train_df.to_json('fine_tuning_train_data.json', orient='records', lines=True)
val_df.to_json('fine_tuning_test_data.json', orient='records', lines=True)
```

- On a chargé un modèle GPT-2 fine-tuné et son tokenizer associé

Ensuite, On passe le modèle en mode évaluation (`model.eval()`), ce qui désactive des fonctionnalités comme le dropout, qui sont utilisées uniquement pendant l'entraînement. Le modèle est ainsi prêt à faire des prédictions ou des évaluations sur de nouvelles données.

✓
0 s

```
from transformers import GPT2LMHeadModel, GPT2Tokenizer
import torch
import json

# Charger le modèle et le tokenizer fine-tunés
model_path = "fine_tuned_gpt2"
model = GPT2LMHeadModel.from_pretrained(model_path)
tokenizer = GPT2Tokenizer.from_pretrained(model_path)
model.eval()
```

```
GPT2LMHeadModel(
  (transformer): GPT2Model(
    (wte): Embedding(50257, 768)
    (wpe): Embedding(1024, 768)
    (drop): Dropout(p=0.1, inplace=False)
    (h): ModuleList(
      (0-11): 12 x GPT2Block(
        (ln_1): LayerNorm((768,)), eps=1e-05, elementwise_affine=True)
        (attn): GPT2SdpaAttention(
```

- Maintenant on génère une réponse à une question donnée en utilisant le modèle GPT-2 et le tokenizer. On commence par encoder la question avec le tokenizer, en ajoutant les tokens spéciaux nécessaires, en appliquant un padding si nécessaire, et en tronquant les séquences trop longues. Ensuite, on utilise la fonction `model.generate()` pour générer une réponse en contrôlant plusieurs paramètres : la longueur maximale de la réponse, la température (qui ajuste la créativité de la réponse), et les méthodes `top_k` et `top_p` qui influencent la diversité des tokens générés. Une fois la réponse générée, elle est décodée pour obtenir un texte lisible, et les tokens spéciaux sont retirés. La réponse finale est ensuite renvoyée sous forme de texte.

```

def generate_answer(question, model, tokenizer, max_length=50, temperature=0.7, top_k=50, top_p=0.9):

    inputs = tokenizer.encode_plus(
        question,
        return_tensors='pt',          # Retourne des tenseurs PyTorch
        add_special_tokens=True,      # Ajoute les tokens spéciaux (debut, fin )
        padding='longest',           # Ajoute du padding si la séquence est courte
        truncation=True               # Tronque la séquence si elle est trop longue
    )

    # Génération de la réponse
    output = model.generate(
        input_ids=inputs['input_ids'],
        attention_mask=inputs['attention_mask'],
        max_length=max_length,
        temperature=temperature,
        top_k=top_k,
        top_p=top_p,
        pad_token_id=tokenizer.eos_token_id # Ou définir explicitement un pad_token_id si nécessaire
    )

    # Décodage et retour de la réponse
    answer = tokenizer.decode(output[0], skip_special_tokens=True)
    return answer

```

- Ici, on teste notre modèle en posant des questions et en recevant des réponses générées par le modèle GPT-2.
Tant que l'utilisateur ne tape pas "exit" ou "quitter", une boucle continue permet de saisir une question, de générer une réponse, et de l'afficher. Lorsque l'utilisateur souhaite quitter, la boucle s'arrête et le programme affiche "Fin du programme".

```

while True:
    # Demande à l'utilisateur de saisir une question
    question = input("Entrez votre question (ou tapez 'exit' pour quitter) : ")

    # Permet de sortir de la boucle si l'utilisateur le souhaite
    if question.lower() in ['exit', 'quitter']:
        print("Fin du programme.")
        break

    # Génération de la réponse à partir de la question saisie
    answer = generate_answer(question, model, tokenizer)

    # Affichage de la réponse générée
    print(f"Réponse générée: {answer}\n")

```

```

↳ Entrez votre question (ou tapez 'exit' pour quitter) : qui enseigne trait aut lang nat
/usr/local/lib/python3.11/dist-packages/transformers/generation/configuration_utils.py:628: UserWarning:
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/transformers/generation/configuration_utils.py:633: UserWarning:
  warnings.warn(
Réponse générée: qui enseigne trait aut lang nat. ?l. lazib enseigne ce cours.

Entrez votre question (ou tapez 'exit' pour quitter) : qui enseigne ihm
Réponse générée: qui enseigne ihm ?m. soualah enseigne ce cours.

Entrez votre question (ou tapez 'exit' pour quitter) : exit
Fin du programme.

```

V. Perspectives :

À l'avenir, plusieurs améliorations sont envisageables :

- **Enrichir le jeu de données** avec davantage d'exemples pour améliorer la précision.
- **Expérimenter avec d'autres modèles** comme GPT-3 ou BERT.
- **Intégrer le système dans une interface utilisateur** sous forme de chatbot accessible aux étudiants.

Ce projet démontre le potentiel des modèles NLP dans le domaine éducatif et ouvre la voie à des applications plus interactives et intelligentes.

VI. Conclusion:

Dans ce projet, nous avons développé un **système de questions-réponses** permettant aux étudiants d'obtenir rapidement des informations sur leurs cours, enseignants et emplois du temps. Nous avons fine-tuné le modèle **GPT-2** sur nos données académiques spécifiques à notre département d'informatique.